

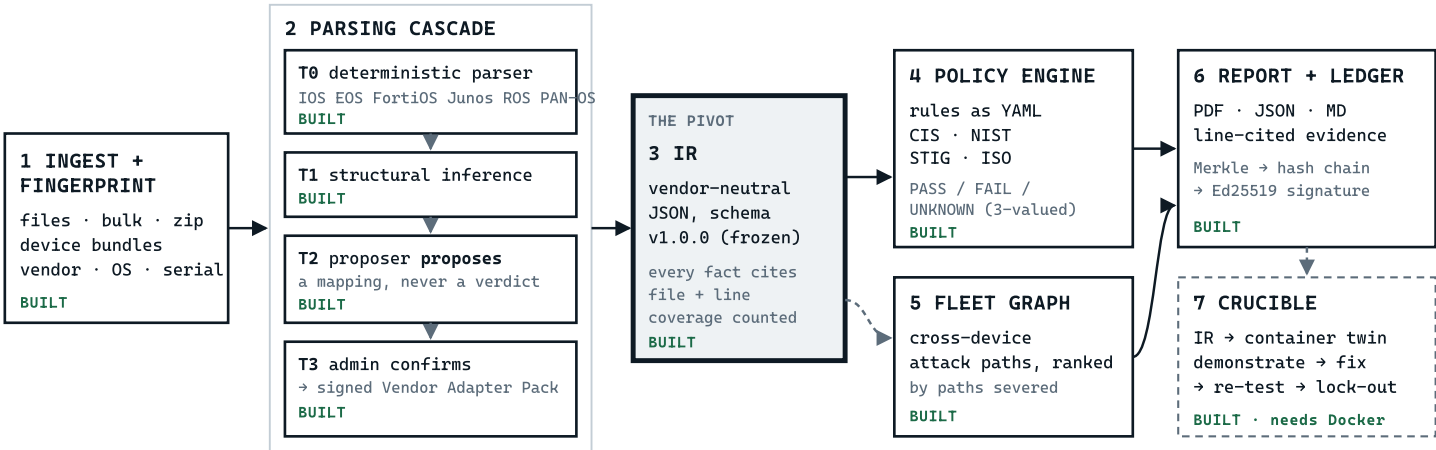
Compliance you can prove. Architecture document.

The problem. A single government network runs firewalls, routers and switches from dozens of vendors, and every device must comply with CIS, NIST SP 800-53, DISA STIG and ISO/IEC 27001. Auditors either work a three-hundred-item checklist by hand or buy a suite locked to one vendor. NTRO asked for a vendor-agnostic engine that an administrator can teach new vendors *without redeploying code*. For a customer like NCIIPC it must also never send a configuration off the machine.

THE PIVOT

Everything turns on one **vendor-neutral intermediate representation (IR)**, an LLVM IR for network security posture. Parsers write into it. Rules, reports, remediation, graph analysis and the verification sandbox read *only* from it. Adding a vendor never touches a rule; adding a framework never touches a parser. That one property satisfies the *no manual code modification* requirement, and it is enforced by an import-contract check, not by convention.

01 PIPELINE



Every stage is built and tested · dashed = needs a Docker daemon, without which findings stay ASSERTED

Unparsed lines fall down the cascade; nothing below stage 3 ever reads raw config text

02 THE FIVE INVARIANTS: GUARANTEES, EACH ENFORCED BY TESTS

INVARIANT	CONSEQUENCE
1 The AI never decides pass or fail. It only writes parsers.	A proposal is a candidate extraction rule, applied deterministically; the verdict comes from the YAML engine. A model is asked only to <i>choose</i> among candidates and point at the value token — never to write a pattern, so it cannot smuggle a regex into a pack.
2 Every finding carries line-level evidence.	File, line number and raw text. This answers “ <i>how do I know the AI didn’t invent this?</i> ” structurally: the IR builder refuses a fact without a source line.
3 Fail closed. Unparsed input can never produce a pass.	A missing fact evaluates to UNKNOWN under Kleene three-valued logic, and UNKNOWN can never become PASS.
4 Coverage is published, not hidden.	Every report states parsed 163 of 164 lines (99.4%) and lists each uninterpreted line verbatim. <code>parsed + unparsed == total</code> is asserted.
5 Three honest states, never two.	DEMONSTRATED (proven on a twin) · ASSERTED (rule matched) · UNKNOWN (could not determine). No certainty is claimed that was not earned.

03 WHY AN IR, AND NOT A PROMPT

The obvious approach pastes a whole config into an LLM and asks for a verdict. That fails every property an audit needs: it is not reproducible, it cannot cite a line, it fails open when it misreads, and it sends the network’s blueprint to someone else’s server.

With an IR, the model’s job shrinks to the one thing it is good at and the one place a mistake is caught: proposing how to *read* a line nobody recognised. Because context limits only ever apply to single unresolved lines, a 40,000-line config costs nothing extra.

04 AIR-GAPPED BY CONSTRUCTION

A device configuration maps every ACL, trust relationship and credential hash. For NCIIPC, a cloud LLM API is a hard disqualifier, whatever the preference. The system makes **zero outbound calls**, with no cloud SDK in the dependency tree. The console ships no web fonts or CDN assets, and a test fails the build if any asset reaches off the host. The default proposer is lexical and needs no model; where one is wanted, the transport itself refuses any address that is not loopback, so a proxy in the environment cannot capture a prompt (ADR 0003). The offline bundle is verified by installing it with no package index and running a full audit from the installed copy.

05 MAPPING TO THE FIVE NTRO COMPONENTS

COMPONENT	WHERE IT LIVES	WHAT IT DOES	STATUS
1 Unified ingestion	ingest/ fingerprint/ console, POST /audit	Single or bulk upload, directories, zip archives and device bundles (config plus <code>show version</code> read as one device). Refuses path traversal, symlinks and zip bombs. Weighted fingerprinting with a published confidence. One unreadable device never aborts the fleet.	BUILT
2 AI training module	training/ adapters/ console	Tiers 1–3: infer structure, propose a field mapping, admin confirms in a low-code console. Exported as a signed, portable Vendor Adapter Pack — data only, a fixed transform library, a trust store that refuses an unknown signer — so one organisation teaches a vendor and every deployment audits it next morning with no code shipped. Measured cold on RouterOS with its own parser held out: 7 of 9 fields .	BUILT
3 Multi-framework engine	policy/ rules/*.yaml	A control is data: an assertion over IR paths, a rationale, framework IDs and per-vendor fixes. Each of the 13 controls maps to CIS v8, NIST SP 800-53, DISA STIG and ISO 27001, and <code>--framework</code> selects without re-parsing. A DISA XCCDF 1.1/1.2 benchmark imports into the same engine through explicit bindings.	BUILT
4 Reporting & PDF	report/ graph/ ledger/	A per-device PDF with identity (vendor, model, OS, serial), pass/fail by severity, line-cited evidence, vendor-specific remediation CLI, a what-if score, a coverage appendix, and a verification hash on every page. Across a fleet, remediation is ranked by the attack paths each fix severs.	BUILT
5 Vendor-agnostic scale	schemas/ir parser registry	A frozen, versioned IR schema. New rules and frameworks need no code — and neither does a new vendor : it is taught in the console and shipped as a signed pack, which is exactly the “no manual code modification” requirement.	BUILT

06 PARSING CASCADE: THE AI PROPOSES, THE ENGINE DECIDES

Tier 0: deterministic parsers record provenance for every fact and account for every line. Unclaimed lines fall to **Tier 1**, which detects the grammar (brace, indent, flat) and builds a generic tree. Unknown nodes reach **Tier 2**, which proposes a mapping such as `set admintimeout 10 → mgmt.idle_timeout_min`. The mapping is applied as a deterministic rule, *never* as a judgement. At low confidence, **Tier 3** asks an administrator and signs the confirmed mapping into an adapter pack, so the next run handles that line at Tier 0. A low-confidence fingerprint runs no parser at all: an unknown device reports UNKNOWN instead of being misread.

07 CRUCIBLE: DEMONSTRATED, NOT ASSERTED

A finding says a device *is* vulnerable. Crucible proves it. The IR renders to a disposable container twin; stdlib probes attempt the thing the rule forbids; the remediation is applied; the probe re-runs; and a final check confirms the fix did not lock the administrator out. Only then is a finding **DEMONSTRATED**, and the proof digest goes into the ledger leaf. A probe that fails to demonstrate leaves the finding **ASSERTED** — never a pass.

08 REMEDIATION THAT CANNOT LOCK YOU OUT

Administrators ignore compliance reports when applying the fixes is scarier than the findings. Fixes are ordered by safety, not severity: **0** establish the safe path (enable SSH) → **1** harden → **2** disable weak services (Telnet off) → **3** restrict management access. Pasted top to bottom, the administrator is never cut off.

11 MEASURED ON THE CURRENT BUILD · precision and recall from crucible validate against hand-labelled configurations

224	1.00	0.90	98.1%
TESTS PASSING	PRECISION · 0 FALSE POSITIVES	RECALL · 0 MISSED AS PASS	MEAN LINE COVERAGE

37 true positives, **no** false positives, fact accuracy 53/53. The four misses landed on UNKNOWN — the auditor is told to look, not that the device is fine — and are counted separately from a false negative. **The labels await human review, so these figures are provisional**, as the tool itself prints.

PHASE 0–1 · DONE	PHASE 2 · DONE	PHASE 3 · DONE	PHASE 4–5 · DONE	OPEN
IR frozen; config in → signed PDF out	training loop, packs, XCCDF, PAN-OS	fleet graph, ranking, accuracy	Crucible twin; drift, offline bundle	60+ real configs; label review